

Last time: El Gamal Encryption

- Gen: let $\mathbb{G}$ be a finite cyclic group of order q and generator g .
Pick $x \in \mathbb{Z}_q$ randomly and set $h = g^x$.

The public key is $pk = (\mathbb{G}, q, g, h)$ and the private key is $sk = x$.

- Enc: on a message $m \in \mathbb{G}$, choose a random $y \in \mathbb{Z}_q$ and output

$$c = (g^y, h^y \cdot m)$$

g^{xy}

- Dec: on a ciphertext $c = (c_1, c_2)$, compute

$$m = c_2 / c_1^x$$

Euler Phi function $\phi(n)$

Positive integers up to n that are relatively prime to n

$$\phi(p) = p - 1 \text{ for prime } p$$

$\phi(N) = (p - 1)(q - 1)$ for **composite number** $N = pq$ where p and q are distinct prime numbers

Euler's Theorem

- $\phi(N)$ is the order of $\mathbb{Z}_N^*$

For any element x of $\mathbb{Z}_N^*$, we have $x^{\phi(N)} \bmod N = 1$

- Example

$$\mathbb{Z}_N^* = \{ 1, 3, 7, 9 \} \qquad \phi(10) = 4$$

$$3^{\phi(10)} \bmod 10 = 3^4 \bmod 10 = 81 \bmod 10 = 1$$

$$7^{\phi(10)} \bmod 10 = 7^4 \bmod 10 = 2401 \bmod 10 = 1$$

$$9^{\phi(10)} \bmod 10 = 9^4 \bmod 10 = 6561 \bmod 10 = 1$$

Related theorems

- Fermat's Little Theorem: Let p be a prime. For all $x \in \mathbb{Z}_p^*$, we have $x^{p-1} = 1 \bmod p$
- For any finite group $\mathbb{G}$ with order q , for any element $x \in \mathbb{G}$,
$$x^q = 1$$

Simplification of modular exponentiation

$$x^a \bmod N = x^{a \bmod \phi(N)} \bmod N$$

$$9^{400} \text{ mod } 10$$

$$= 9^a \bmod \phi(10) \bmod 10$$

$$= 9^1 \bmod 10$$

= 9

Factoring assumption

Given $N = pq$ where p and q are large prime numbers, it is computationally hard to find p and q

$O(N) = O(2^l)$ naively, $O(\sqrt{N}) = O(2^{l/2})$, $O(\sqrt[3]{N}) = O(2^{l/3})$ best known algorithm

Factoring assumption

Experiment:

- Run $\text{Gen}(1^n)$ to get (N, p, q)
- A is given N , and outputs p', q'
- Output 1 if $p'q' = N$

$\text{Gen}(1^n)$

- **Prime number theorem:** probability that a random integer $\leq N$ is prime is very close to $1 / \log(N)$. Sample p randomly with the desired bitlength
- **Primality testing:** (Miller-Rabin algorithm Chapter 9.2.2)
- $N = pq$

RSA (1975-1978)

- Developed shortly after Diffie-Hellman paper
- Takes its name from the initials of its inventors
 - Ron **R**ivest
 - Adi **S**hamir
 - Leonard **A**delman
- Possibly best known public key algorithm



Shamir, Rivest and Adelman ↑

From: <http://www.usc.edu/dept/molecular-science/RSApics.htm>

- Clifford Cocks (w/ James Ellis and Malcolm Williamson), at GCHQ (UK), invented independently a particular case of this method 3 years before RSA, but it was classified by British intelligence
 - Declassified in 1997

Key generation

1. Choose two large prime numbers p and q such that $p \neq q$, randomly and independently of each other. Set $N = pq$.

2. Pick integer e **coprime** with $\phi(N) = (p - 1)(q - 1)$

$$\gcd(e, (p - 1)(q - 1)) = 1$$

3. Compute d such that

$$e \cdot d = 1 \bmod (p - 1)(q - 1)$$

Public key $pk = (N, e)$

Private key $sk = d$

Permutation

- e coprime with $\phi(N) \Rightarrow \exists d: e \cdot d = 1 \bmod (p-1)(q-1)$
- For any element $x \in \mathbb{Z}_N^*$, $f(x) = x^e \bmod N$ is a permutation and $f'(y) = y^d \bmod N$ is its inverse.
 - $$\begin{aligned} f'(f(x)) &= (x^e)^d \bmod N \\ &= x^{ed \bmod \phi(N)} \bmod N \\ &= x^1 \bmod N = x \end{aligned}$$

RSA Encryption and Decryption

Public key $pk = (N, e)$ $N = pq$

Private key $sk = d$ such that $e \cdot d = 1 \bmod (p - 1)(q - 1)$

Message space: $m \in \mathbb{Z}_N^*$

- Encryption: $c = m^e \bmod N$
- Decryption: $m = c^d \bmod N$

RSA Cryptosystem

- Gen:
 - $N = pq$, with p and q primes
 - e relatively prime to $\phi(N) = (p - 1)(q - 1)$
 - d inverse of $e \bmod \phi(N)$
- Keys:
 - Public key: $pk = (N, e)$
 - Private key: $sk = d$
- Encryption:
 - Plaintext m in $\mathbb{Z}_N^*$
 - $c = m^e \bmod N$
- Decryption:
 - $m = c^d \bmod N$

Example:

- Gen:
 - ♦ $p = 7, q = 17$
 - ♦ $N = 7 \cdot 17 = 119$
 - ♦ $\phi(N) = 6 \times 16 = 96$
 - ♦ $e = 5$
 - ♦ $d = 77$
- Keys:
 - ♦ public key: (119, 5)
 - ♦ private key: 77
- Encryption:
 - ♦ $m = 19$
 - ♦ $c = 19^5 \bmod 119 = 66$
- Decryption:
 - ♦ $m = 66^{77} \bmod 119 = 19$

Why RSA works: correctness

- $ed \bmod (p-1)(q-1) = 1$

- $N = pq$

- $(m^e \bmod N)^d \bmod N$
 - $= m^{ed} \bmod N$
 - $= m^{ed \bmod (p-1)(q-1)} \bmod N$
 - $= m \bmod N$

← Euler's theorem

Why RSA works: security

- Factoring assumption: given $N = pq$ where p and q are large prime numbers, it is computationally hard to find p and q
- RSA assumption:
 1. Run $\text{Gen}(1^n)$ to obtain (N, e, d)
 2. Choose a random $y \in \mathbb{Z}_N^*$
 3. A is given (N, e, y) and outputs $x \in \mathbb{Z}_N^*$
 4. Output 1 if $x^e = y \bmod N$

$$\Pr[\text{RSA}_A(n) = 1] \leq \text{negl}(n)$$

- Gen:
 - $N = pq$, with p and q primes
 - e relatively prime to $\phi(N) = (p - 1)(q - 1)$
 - d inverse of $e \bmod \phi(N)$
- Keys:
 - Public key: $pk = (N, e)$
 - Private key: $sk = d$
- Encryption:
 - Plaintext m in $\mathbb{Z}_N^*$
 - $c = m^e \bmod N$
- Decryption:
 - $m = c^d \bmod N$

Security

- RSA Security factoring challenge (discontinued)
- In 1999, 512-bit challenge factored in 4 months using 35.7 CPU-years
- In 2005, a team of researchers factored the RSA-640 challenge number using 30 2.2GHz CPU years
- In 2004, the prize for factoring RSA-2048 was \$200,000

https://en.wikipedia.org/wiki/RSA_Factoring_Challenge

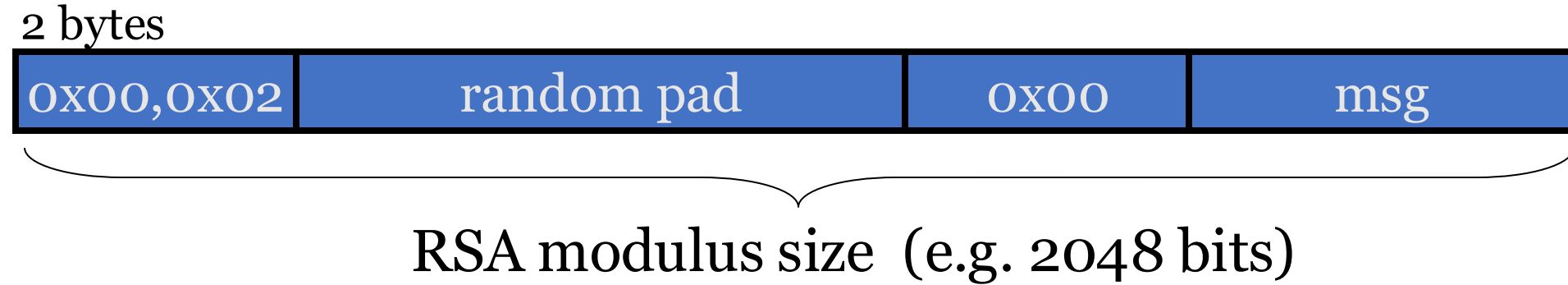
Is plain RSA CPA-secure?

- Gen:
 - $N = pq$, with p and q primes
 - e relatively prime to $\phi(N) = (p - 1)(q - 1)$
 - d inverse of $e \bmod \phi(N)$
- Keys:
 - Public key: $pk = (N, e)$
 - Private key: $sk = d$
- Encryption:
 - Plaintext m in $\mathbb{Z}_N^*$
 - $c = m^e \bmod N$
- Decryption:
 - $m = c^d \bmod N$

Padded RSA

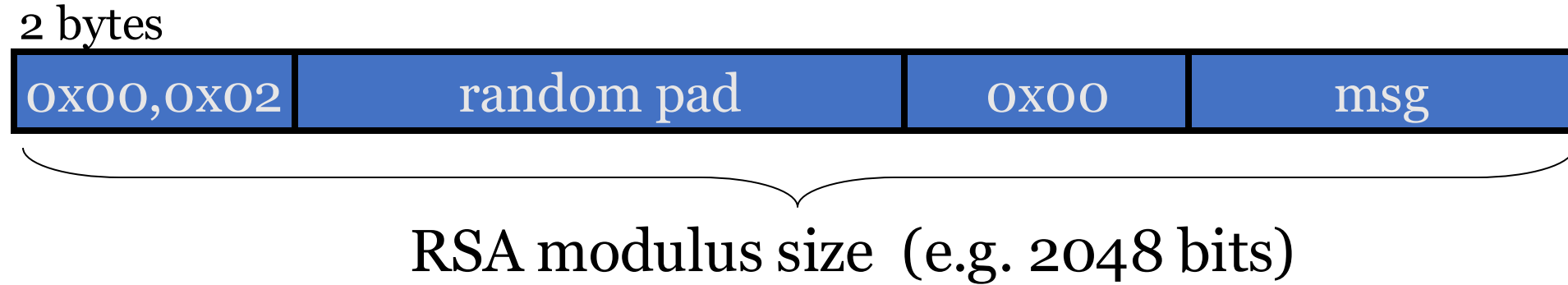
- Gen: (unchanged)
 - $N = pq$, with p and q primes. e relatively prime to $\phi(N) = (p - 1)(q - 1)$
 - d inverse of $e \bmod \phi(N)$
 - Public key: $pk = (N, e)$, Private key: $sk = d$
- Encryption:
 - Plaintext $m \in \{0,1\}^{\ell_1}$, choose $r \in \{0,1\}^{\ell_2}$ uniformly such that $\ell_1 + \ell_2 = \log N$
 - Interpret $m' = r || m$ as an element in $\mathbb{Z}_N^*$, $c = m'^e \bmod N$
- Decryption:
 - $m' = c^d \bmod N$, output the last ℓ_1 bits of m'

RSA in practice: PKCS#1 v1.5



- Resulting value is RSA encrypted
- Widely deployed, e.g. in HTTPS

RSA in practice: PKCS#1 v1.5



- Minimum length of r is only 8 bytes in PKCS#1 v1.5, which is not secure enough
- r of the same length as the message is recommended.
- It is conjectured to be CPA secure, but no formal proof based on RSA assumption

What about longer messages?

Chosen Ciphertext Attack (CCA) security

Experiment $\text{PubK}_{A,\Pi}^{\text{CCA}}(n)$:

1. Key (pk, sk) is generated using $\text{Gen}(1^n)$
2. Adversary A is given input 1^n , pk and $\text{Dec}_{sk}(\cdot)$. A outputs two messages $m_0, m_1 \in \mathcal{M}$ s.t. $|m_0| = |m_1|$
3. A random bit $b \in \{0,1\}$ is chosen. $c = \text{Enc}_{pk}(m_b)$ is given to A .
4. A continues to have **oracle access** $\text{Dec}_{sk}(\cdot)$, but is not allowed to query $\text{Dec}_{sk}(c)$. A outputs b'
5. Experiment outputs 1 if $b=b'$, and 0 otherwise.

Definition: a public-key encryption $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ is CCA-secure, if for all probabilistic polynomial time adversaries A , there is a negligible function negl such that for all n ,

$$\Pr[\text{PubK}_{A,\Pi}^{\text{CCA}}(n) = 1] \leq \frac{1}{2} + \text{negl}(n)$$

Symmetric-key vs. Public-key crypto

Symmetric crypto (AES)

- Need shared secret
- 128-bit key in AES
- 1,000,000 encryptions/s on a 1 GHz processor
- >100x speedup in hardware

Public-key crypto (RSA)

- Need authenticated public key
- 2048-bit key in RSA encryption
- 100 encryptions/s on 1 GHz processor
- ~ 10x speedup in hardware

Key Encapsulation Mechanism (KEM)

